

2013 年参考答案

1. 顺序表

(1) 静态查找表存储结构

```
typedef struct{
    ElemType *elem;
    int Tablelen;
}SSTable
```

(2) 顺序查找过程

```
(2) int Search_Seq(SSTable ST, ElemType key)
{
    int i
    ST.elem[0].key=key;
    for(i=ST.Tablelen; ST.elem[i].key!=key; --i)
    {
        return i;
    }
}
```

(3) 平均查找长度

$$(1+2+3+\dots+n)/n=(n+1)/2$$

2. 直接插入排序

(1) 过程:

第一趟: 38 49 65 97 76 13 27 49

第二趟: 38 49 65 97 76 13 27 49

第三趟: 38 49 65 97 76 13 27 49

第四趟: 38 49 65 76 97 13 27 49

第五趟: 13 38 49 65 76 97 27 49

第六趟: 13 27 38 49 65 76 97 49

第七趟: 13 27 38 49 49 65 76 97

(2) 存储结构及算法


```

typedef struct
{
    KeyType key; //关键字项
    OtherType data; //其他数据项
}RecordType; //记录类型

void D_Insert(RecordType R[], int n)
{
    int i,j;
    for(i=2; i<=n; i++)//进行n-1趟排序
    {
        if(R[i].key < R[i-1].key)//小于时,需将R[i]插入有序表
        {
            R[0] = R[i]; //设置监视哨并保存插入记录
            R[i] = R[i-1];
            for(j=i-2; R[0].key < R[j].key; j--)
                R[j+1] = R[j]; //将关键字大与R[i].key(此时为R[0].key)的所有记录顺序后移一位
            R[j+1] = R[0]; //将R[i](此时为R[0])插入相应位置
        }
    }
}

```

3. (1) 结点均含大小相同的空闲块。

(2) 建立若干个可利用空间表, 同一链表中的结点大小相同, 不同链表中结点大小, 一般成倍数关系。

(3) 结点所包含的空闲块的大小可随请求而变。

4. 存储结构有四种: 邻接矩阵法, 邻接表法, 十字链表法, 邻接多重表法, 存储结构的描述可看教材定义。(注意: 存储结构的定义不是考察的重点, 重点是操作)

5. 线性表

(1) 线性表的动态分配顺序存储结构

```

#define LIST_INIT_SIZE 100
#define LISTINCREMENT 10
typedef struct{
    ElemType *elem;
    int length;
    int listsizes;
}SqlList;

```

(2) 顺序表初始化算法


```

bool InitList_Sq(SqList &L)
{
    L.elem=(ElemType*)malloc(LIST_INIT_SIZE*sizeof(ElemType));
    if(!L.elem)
        exit(OVERFLOW);
    L.length=0;
    L.listsize=LIST_INIT_SIZE;
    return true;
}

```

(3) 插入算法

```

bool ListInsert_Sq(SqList &L,int i,ElemType e)
{
    if(i<1||i>L.length+1)
        return ERROR;
    if(L.length>=L.listsize)
    {
        newbase=(ElemType*)realloc(L.elem,(L.listsize+LISTINCREMENT)*sizeof(ElemType));
        if(!newbase)
            exit(OVERFLOW);
        L.elem=newbase;
        L.listsize+=LISTINCREMENT;
    }
    q=&(L.elem[i-1]);
    for(p=&(L.elem[L.length-1]);p>=q;--p)
        *(p+1)=*p;
    *q=e;
    ++L.length;
    return true;
}

```

6.链表

(1) 存储结构

```

#define Maxsize 50
typedef struct{
    ElemType data[Maxsize];
    int top;//栈顶指针指向当前栈顶的位置
}SqStack;

```

(2) 入栈

```
bool Push(SqStack &S, ElemType x)
{
    if(S.top == Maxsize - 1)
        return false;
    S.data[++S.top] = x;
    return true;
}
```

(3) 出栈

```
bool Pop(SqStack &S, ElemType &x)
{
    if(S.top == -1)
        return false;
    x = S.data[S.top--];
    return true;
}
```

7. 串联接：将两个字符串合并为一个


```

seqString *strconcat(seqString s,seqString T)
{
    int i;
    seqString *r;
    if(s.length+T.length>=Maxsize)
    {
        printf("can not concat\n");
        return null;
    }
    r=(seqString *)malloc(sizeof(seqString));
    for(i=0;i<s.length;i++)
    {
        r->str[i]=s.str[i];
    }
    for(i=0;i<T.length;i++)
    {
        r->str[s.length+i]=T.str[i];
    }
    r->length=s.length+T.length;
    r->str[r->length]='\0';
    return r;
}

```

8. (可不看, 省略)

9. 二叉树

(1) 存储结构

```

typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;

```

(2) 有三种遍历方法:

先序遍历: 根左右 (NLR)

中序遍历: 左根右 (LNR)

后序遍历: 左右根 (LRN)

(4) 中序遍历算法: 遍历算法中的递归思想非常重要, 也很方便使用, 比如计算结点个数, 深度等等。

```
void InOrder(BiTree T )
{
    if(T!=null)
    {
        InOrder(T->lchild);
        visit(T);
        InOrder(T->rchild);
    }
}
```

2014 年参考答案

1.算法：是对特定问题求解步骤的一种描述；

特性：有穷性，确定性，可行性，输入，输出

注意：区分“好”的算法特性：

正确性，可读性，健壮性，效率与低存储量需求

2.双链表

(1) 存储定义

```
typedef struct DNode{
    ElemType data;
    struct DNode *prior,*next;
}DNode,*DLinklist;
```

(2) 插入关键代码

```
s->next=p->next;
p->next->prior=s;
s->prior=p;
p->next=s;
```

(3) 删除关键代码

```
p->next=q->next;
q->next->prior=p;
free(q);
```

3.队列：是一种操作受限制的线性表，只允许在表的一端进行插入，而在另一端进行删除。

(1) 存储结构描述

```
typedef struct{
    ElemType data;
    struct LinkNode *next;
}LinkNode;
typedef struct{
    LinkNode *front,*rear;
}LinkQueue;
```

(2) 初始化

```
void InitQueue(LinkQueue &Q)
{
    Q.front=Q.rear=(LinkNode *)malloc(sizeof(LinkNode));
    Q.front->next=null;
}
```

(3) 销毁队列

```
void distoryQueue(seQueue &Q)
{
    while(Q.front!=Q.rear)
    {
        free(&Q.base[Q.front]); // 释放空间
        Q.front=(Q.front+1)%MaxSize;
    }
}
```

(4) 入队

```
void EnQueue(LinkQueue &Q,ElemType x)
{
    s=(LinkNode *)malloc(sizeof(LinkNode));
    s->data=x;
    s->next=null;
    Q.rear->next=s;
    Q.rear=s;
}
```

4-6:不是考察的重点，后面没有考察过，可以不用看

7. 顺序表的有序插入

//思路: 从最后一个元素开始, 如果当前元素值大于x, 该元素就后移, 直到找到第一个小于x的元素, x放入到该元素之后

```
void Insert_SeqList_Order(SeqList *L, int x)
```

```
{
    int k, n=L->len;
    if(n == MAXSIZE - 1)
        printf("表满!");
    else
    {
        for(k=n; k>=1; k--)
            if(x < L->data[k])
                L->data[k+1] = L->data[k];
            else
                break;
        L->data[k+1] = x;
        L->len++;
    }
}
```

8. 递归

```
int f(int n)
{
    if(n==0)
        return n+1;
    else
        return n*f(n/2);
}
```

非递归

```
int f(int n)
{
    int sum=1;
    while(n!=0)
    {
        sum*=n;
        n/=2;
    }
    return sum;
}
```

9. 折半插入

//查找成功返回下标, 否则返回-1

int BinSearch(RecordType R[], int n, int k)

{

int low=0, high=n-1, mid;

while(low<=high)//查找区间不为空

{

//mid取该查找区间的中间记录位置

mid = low + (high-low)/2;//结果和(low+high)/2相同, 这样做可以避免low+high太大导致溢出

if(R[mid].key==k)//当中间记录的关键字与k相等时

return mid;//查找成功

else

if(R[mid].key>k)

high=mid-1;//继续在R[low]~R[mid-1]中查找

else

low=mid+1;//继续在R[mid+1]~R[high]中查找

}

return -1;

}

10. 二叉树范密度

(1) 求树的深度:

int treeDepth(Bitree T)

{

if(T)

return max(treeDepth(T->lchild), treeDepth(T->rchild))+1;

else

return 0;

}

(2) 二叉树各层结点数的最大值:


```
int maxLayer(BiTree T)
{
    if(T==null) return 0;
    Queue Q;
    InitQueue(Q);
    int max=0,pre=1,h=0,i=0;
    BiTree P=T;
    EnQueue(Q,P);
    while(Q.front!=Q.rear){
        while(i<pre){
            DeQueue(Q,P);
            if(P->lchild){
                h++;
                EnQueue(Q,P->lchild);
            }
            if(P->rchild){
                h++;
                EnQueue(Q,P->rchild);
            }
            i++;
        }
        if(h>max){
            max=h;
        }
        pre=h;
        i=0;
        h=0;
    }
    return max;
}
```